CHECKOUT GUARDIAN

System Design Document (SDD) — Version 1.0

Project: Checkout Guardian

Author: Architecture & Engineering Team

Status: Approved / Implementation-Ready

Date: October 2026

Table of Contents

1. Introduction
2. System Architecture Overview
3. Architectural Design Decisions
4. Component-Level Design
5. Data Flow Architecture
6. Detailed Sequence Diagrams
7. Database Design
8. API and Interface Design
9. AI Pipeline Architecture
10. Merchant Identity & Offer Consistency Engine Design
11. Risk Scoring and Decision Architecture
12. PayPal Integration Design
13. Security Architecture
14. Error Handling and Resilience Design
15. Deployment Architecture
16. Project Directory and Code Organisation
17. Testing Architecture
18. Design Decisions, Trade-offs and Limitations
19. Implementation Mapping and Team Allocation
20. Final System Design Summary

1. Introduction

Purpose: This System Design Document (SDD) serves as the definitive architectural blueprint for implementing Checkout Guardian for the PayPal AI Hackathon 2026. It bridges product requirements (PRD), technical specifications (TRD), and software behavior (SRS) into an actionable engineering specification.

Project Overview: Checkout Guardian is an AI-powered buyer-protection system designed to identify fraudulent e-commerce websites, brand impersonation, deceptive pricing, and mismatches between the storefront merchant and the actual PayPal payment recipient.

Central Innovation: The *Merchant Identity & Offer Consistency Engine*, which cross-references storefront claims against payment gateway beneficiary metadata prior to transaction capture.

2. System Architecture Overview

The system is architected as a robust modular monolith featuring a React SPA / Chrome Extension frontend, an Express.js orchestration backend, a dedicated AI and consistency analysis pipeline, PostgreSQL persistence storage, and secure integration with the PayPal Sandbox APIs.

3. Architectural Design Decisions

Design Choice	Selected Architecture	Justification
Architecture Style	Modular Monolith	Optimized for a four-member hackathon team; avoids unnecessary microservices overhead while maintaining strict module boundaries.
Backend Runtime	Node.js & Express.js	Asynchronous I/O, rapid API prototyping, and full-stack JavaScript uniformity.
Database	PostgreSQL	Strong ACID guarantees and structured relational integrity for audit logs, transactions, and risk assessments.
AI Processing	External LLM API via Node.js Service	Leverages cloud-based semantic intelligence while enforcing deterministic rule gating before payment decisions.

4. Component-Level Design

- **4.1 Frontend Module:** React SPA and Chrome Extension managing UI states, risk warning modals, and dashboard analytics.
- **4.2 Website Authenticity Module:** Extracts URL metadata, WHOIS creation age, and SSL certificate health.

- **4.3 Merchant Identity Module:** Ingests footer text and corporate disclaimers to verify business registration claims.
- **4.4 Product & Price Intelligence Module:** Compares advertised pricing against baseline market norms to flag predatory discounts.
- **4.5 Merchant Identity & Offer Consistency Engine:** Reconciles storefront branding claims against the actual PayPal `PAYEE_MERCHANT_ID`.
- **4.6 AI Analysis Module:** Executes structured prompt extraction with strict JSON schema validation.
- **4.7 Risk Assessment Engine:** Aggregates heuristic scores and consistency flags into LOW, MEDIUM, or HIGH risk tiers.
- **4.8 PayPal Integration Module:** Governs sandbox order creation, customer approval, and backend capture gating.

5. Data Flow Architecture

Information flows unidirectionally from client-side DOM ingestion $\rightarrow$ backend heuristic evaluation $\rightarrow$ PayPal order initialization $\rightarrow$ payee consistency check $\rightarrow$ risk engine verdict $\rightarrow$ conditional capture execution.

6. Detailed Sequence Diagrams

Sequence workflows cover genuine store verification, suspicious offer warnings, brand impersonation detection, and high-risk payment capture pausing within PayPal Sandbox.

7. Database Design

The PostgreSQL relational schema includes tables for `users`, `websites`, `merchants`, `assessments`, `risk_indicators`, `transactions`, and `audit_logs`, enforcing strict data minimization and zero raw PII storage.

8. API and Interface Design

Standardized REST API endpoints follow `/api/v1/` versioning conventions, utilizing JSON payloads and Bearer token authentication where required.


```
{
  "assessmentId": "risk_554433",
  "riskScore": 88,
  "riskTier": "HIGH",
  "action": "PAUSE_CAPTURE",
  "explanations": ["Merchant identity mismatch detected between store brand and PayPal payee."
}
```


9. AI Pipeline Architecture

Combines prompt engineering with Zod schema validation to ensure LLM outputs conform to expected JSON structures. The AI acts solely as an analytical assistant; final payment authorization is governed exclusively by deterministic backend logic.

10. Merchant Identity & Offer Consistency Engine Design

The core engine compares claimed store names against verified domain and PayPal payee IDs. Discrepancies generate weighted mismatch flags that feed directly into the risk scoring engine.

11. Risk Scoring and Decision Architecture

Calculates a weighted risk score (0–100) mapping to three distinct operational tiers: LOW RISK (Allow), MEDIUM RISK (Warn), and HIGH RISK (Pause Capture).

12. PayPal Integration Design

Utilizes the PayPal JavaScript SDK and Orders API in Sandbox. The backend enforces a strict capture gate: if a high-risk mismatch is detected, capture execution is bypassed and a security intervention modal is displayed.

13. Security Architecture

Enforces TLS 1.3 encryption, environment-secured API secrets, parameterized SQL queries against injection, and strict CORS boundaries.

14. Error Handling and Resilience Design

Transient intelligence timeouts invoke fail-open fallback behavior with neutral scoring, preventing false-positive blocking during network disruptions.

15. Deployment Architecture

Containerized local and cloud deployment architecture utilizing Vercel/Render for frontend/backend hosting and managed PostgreSQL instances.

16. Project Directory and Code Organisation

```
checkout-guardian/
├── backend/ (Express app, controllers, services)
├── frontend/ (React SPA & dashboard)
├── extension/ (Manifest V3 Chrome extension)
└── shared/ (Common types and validation schemas)
```

17. Testing Architecture

Comprehensive testing strategy combining Jest unit tests, Supertest API integration checks, and live PayPal Sandbox end-to-end scenario validation.

18. Design Decisions, Trade-offs and Limitations

Focuses on a controlled MVP scope suitable for hackathon execution, prioritizing reliability and core consistency engine verification over generalized web crawling.

19. Implementation Mapping and Team Allocation

Team Member	Assigned Workstream
Member 1	AI & Website Intelligence Module
Member 2	Merchant Verification & Risk Engine
Member 3	Backend API & PayPal Sandbox Integration
Member 4	Frontend SPA & Chrome Extension

20. Final System Design Summary

Checkout Guardian successfully unifies AI semantic extraction, deterministic risk scoring, and payment gateway capture control into a cohesive, production-ready architecture designed to protect digital commerce.

Checkout Guardian — PayPal AI Hackathon 2026 | System Design Document